# Exploring Twitter System Design - Part 4

DAY 108
26/10/2025

## Search Service

- Sometimes traditional DBMS are not performant enough, we need something which allows us to store, search, and analyse huge volumes of Data quickly and in near real time and give results within milliseconds. Elastic Search can help us with this use case.

- Elastic Search is a distributed, free and open search and analytics Engine for all types of Data, including textual, numerical, geospatial, structured, and unstructured. It is Built on top of Apache Lucene

## How do we identify trending topics ?

- Trending functionality will be based on top of the search function
- We can cache the most frequently searched queries, hash tags, and topics in the last N seconds and update them every M seconds using some sort of batch job Mechanism
- Our ranking algorithm can also be applied to the trending topics to give them more weight and personalize them for the user.

## Notifications Service

- Push Notifications are an integral part of any Social Media platform
- We can use a message Queue or a message broker such as Apache Kafka with the notification service to dispatch requests to firebase cloud Messaging (FCM) or Apple Push Notification Service (APNS) which will handle the delivery of the push notifications to user devices.

## Architecture overview

```
[Mobile/Web]
    │  HTTPS / HTTP2 + TLS
    ▼
[API Gateway / Edge] ──> [Auth] ──> [Rate Limiter] ──> [Service Mesh]
    │                                                      │
    │                                                      ├── Tweet Service
(write path)
    │                                                      ├── Engagement
Service (likes/retweets/replies)
    │                                                      ├── Social Graph
Service (follow edges)
    │                                                      ├── Timeline
Service (home/profile)
    │                                                      ├── Search Service
(query/index)
    │                                                      ├── Media Service
(upload/origin)
    │                                                      ├── Notification
Service (push/email)
    │                                                      └── User Service
(profiles, identity)
    │
    ├─► CDN (images/video, thumbnails)
    └─► WebSocket/SSE fanout (live updates)
```

## Core Data plane

- Caches ( Redis / Memcache clusters ) in front of stores
- Event Bus ( Kafka / NATS ) for write logs & fanout
- OLTP stores ( postgres shards / Manhattan )
- Search index ( Lucene / Elastic Search - like "Earlybird" tiers)
- object storage ( S3 / GCS + lifecycle + Glacier / Archive )
- stream Processing ( Heron / Flink / Spark streaming ) for ranking features, trends, counters.
- Batch ( spark / Presto ) for analytics / training

## Write path (Tweet / create)

- clients sends POST / tweets (text + media Refs)
- Auth & rate-limit checked, Snowflake ID allocated
- Persist tweet to Write-Ahead Log (Event bus) and primary store (ack on quorum)
- Publish Events : TweetCreated {tweetID, authorID, entities} to kafka
- Async Consumers :
    - Fan-out to home timelines (push model for normal users)
    - Index into Search
    - counters / metrics (trends, author state)
    - Notification to mentions / replies.
- Return 200 + tweet payload quickly (Target < 150 ms at p99